

Part 2 Design

Database structure for products, warehouses, suppliers, bundles, and inventory history.

Project	Inventory Management System for B2B SaaS
Section	Part 2 - Database Design
Focus	Schema, constraints, relationships, and open product questions

Design approach

The schema below is designed around three practical needs: products may exist in multiple warehouses, stock changes need an auditable history, and suppliers should support reordering. A few requirements are still open, so the design also calls out the questions that should be resolved before final implementation.

Proposed schema

```
CREATE TABLE companies (  
  id          SERIAL PRIMARY KEY,  
  name        VARCHAR(255) NOT NULL,  
  email       VARCHAR(255) UNIQUE NOT  
  NULL, created_at TIMESTAMPTZ NOT  
  NULL DEFAULT NOW()  
);
```

```
CREATE TABLE warehouses (  
  id          SERIAL PRIMARY KEY,  
  company_id  INT NOT NULL REFERENCES companies(id) ON  
  DELETE CASCADE, name    VARCHAR(255) NOT NULL,  
  location    TEXT,  
  is_active   BOOLEAN NOT NULL  
  DEFAULT TRUE, created_at  
  TIMESTAMPTZ NOT NULL DEFAULT  
  NOW()  
);
```

```
CREATE TABLE suppliers (  
  id          SERIAL PRIMARY KEY,  
  name        VARCHAR(255) NOT NULL,  
  contact_email  
  VARCHAR(  
  255), contact_phone  
  VARCHAR(  
  50), website  
  VARCHAR(  
  50)
```

```

255),
created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE products (
  id              SERIAL PRIMARY KEY,
  company_id      INT NOT NULL REFERENCES companies(id) ON
DELETE CASCADE, supplier_id      INT REFERENCES suppliers(id) ON
DELETE SET NULL,
  name            VARCHAR(255) NOT NULL,
  sku             VARCHAR(100) NOT NULL,
  description      TEXT,
  price           NUMERIC(12, 2) NOT NULL CHECK (price >= 0),
  product_type     VARCHAR(50) NOT NULL DEFAULT
'standard', low_stock_threshold INT NOT NULL DEFAULT
10,
  is_active        BOOLEAN NOT NULL DEFAULT
TRUE, created_at  TIMESTAMPTZ NOT NULL
DEFAULT NOW(), updated_at  TIMESTAMPTZ NOT
NULL DEFAULT NOW(),
  UNIQUE(company_id, sku)
);

CREATE TABLE bundle_items (
  bundle_product_id      INT NOT NULL REFERENCES products(id) ON
DELETE CASCADE, component_product_id      INT NOT NULL REFERENCES
products(id) ON DELETE RESTRICT, quantity INT NOT NULL CHECK
(quantity > 0),
  PRIMARY KEY (bundle_product_id,
component_product_id), CHECK (bundle_product_id !=
component_product_id)
);

CREATE TABLE inventory (
  id              SERIAL PRIMARY KEY,
  product_id      INT NOT NULL REFERENCES products(id) ON
DELETE CASCADE, warehouse_id      INT NOT NULL REFERENCES
warehouses(id) ON DELETE CASCADE, quantity      INT NOT NULL
DEFAULT 0 CHECK (quantity >= 0),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(product_id, warehouse_id)
);

CREATE TABLE inventory_movements (
  id              SERIAL PRIMARY KEY,
  inventory_id     INT NOT NULL REFERENCES inventory(id) ON
DELETE CASCADE,
  movement_type   VARCHAR(50) NOT
NULL, INT NOT
quantity_delta    NULL,
  reference_id     INT,
  notes            TEXT

```

```
); created_at      T,
                        TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
CREATE TABLE purchase_orders (
  id          SERIAL PRIMARY KEY,
  company_id  INT NOT NULL REFERENCES
  supplier_id companies(id), INT NOT NULL
  status      REFERENCES suppliers(id),
  ordered_at  VARCHAR(50) NOT NULL DEFAULT
  expected_at 'pending', TIMESTAMPTZ NOT NULL
  received_at DEFAULT NOW(), TIMESTAMPTZ,
  notes       TIMESTAMPTZ,
);
```

```
CREATE TABLE purchase_order_items (
  id          SERIAL PRIMARY KEY,
  purchase_order_id INT NOT NULL REFERENCES purchase_orders(id) ON
  DELETE CASCADE, product_id      INT NOT NULL REFERENCES
  products(id),
  quantity_ordered INT NOT NULL CHECK (quantity_ordered
  > 0), quantity_received INT NOT NULL DEFAULT 0,
  unit_cost        NUMERIC(12, 2)
);
```

Key design choices

- **NUMERIC for price:** Floating point types can introduce rounding errors, which is risky when dealing with financial data. NUMERIC ensures exact precision and avoids issues like storing 19.999999 instead of 20.00.
- **Inventory as a junction table:** There are two main reasons. First, it provides a complete history of inventory changes, which helps in debugging and resolving disputes. Second, it allows calculation of sales velocity, which is required for features like low-stock alerts.
- **Inventory movements:** If a supplier is deleted, the associated products should still remain in the system. Setting supplier_id to NULL preserves the product while removing the supplier relationship. Using RESTRICT would prevent deletion entirely, which may not be desirable.
- **Unique SKU per company:** I assumed that SKU uniqueness is scoped per company rather than globally. Different companies may use the same SKU for different products. However, this assumption should be confirmed with the product team.

Questions to resolve with the product team

- **Are suppliers global or company-specific?**
If suppliers belong to individual companies, the schema would need to include company_id in the suppliers table.
- **Can a product have multiple suppliers?**
If yes, a many-to-many relationship would be required instead of a single supplier_id field.

- **How are bundles handled in inventory?**
It needs clarification whether bundles reduce stock from their components or maintain separate stock levels.
- **Do we need multi-currency support?**
If companies operate internationally, pricing may need to include currency information.
- **Should inventory be allowed to go negative?**
Some systems allow backorders, which would require removing the non-negative constraint.
- **What defines “recent sales activity”?**
The time window (7 days, 30 days, etc.) should be clearly defined, as it affects stock predictions.
- **Should warehouses be soft-deleted or hard-deleted?**
Currently, warehouses use an is_active flag, but cascading deletes on inventory could remove important historical data. This needs alignment